

Lavoisier

Finding engineering errors before anything is built

Tony Walmsley

October 2026

The problem

- Plans and specifications are written in prose
- Prose hides contradictions: double-booked staff, missing parts, waste with nowhere to go
- Those errors surface late — in procurement, on the shop floor, in integration
- The later the find, the higher the cost of the fix

The idea

- Treat an engineering plan like a set of accounts
- Every resource — material, energy, time, money — must balance
- Nothing appears from nowhere; nothing silently disappears
- Named for Lavoisier: *“Nothing is lost, nothing is created, everything is transformed”*
- Then have a computer audit the books — automatically, on every change

How it works, without the jargon

- The plan is written as a precise, machine-checkable **model**
- The checker is the compiler of a programming language (Rust) — software's strictest proof-reader
- We are not building software; we borrow its auditor
- A plan with a contradiction simply **refuses to compile** — with a message saying what is wrong, and where

The hypothesis

- If every *thing* in the plan is a type,
- and every *work step* is a function whose inputs and outputs must balance,
- then an impossible plan cannot pass the checker —
- so whole classes of errors are caught **at the design stage**, before anything is bought, built or scheduled

What gets caught — examples

- A technician booked on two jobs at once
- A step that uses a part no earlier step produced
- Material or energy that goes missing between steps
- Waste with no disposal route
- An overspent time budget or account — overspending *cannot pass the checker*
- An unqualified operator; a machine guard not fitted
- Paying a vendor the wrong amount

Requirements you can audit

- Requirements are numbered (REQ-001, REQ-002, ...) and live **inside the model**
- When one is violated, the error is phrased in the requirement's own words
- A one-command report shows, for every requirement: where it is defined, what satisfies it, and which tests verify it
- A requirement without a verifying test **fails the gate** — nothing ships untested

The evidence so far

- 12 controlled experiments — every design rule is backed by measured evidence
- 19 agreed rules; 54 documented findings, including the honest limitations
- A working library and two complete models (a workshop assembly; making a pot of tea)
- Over 100 automated checks run on every change, end to end

What it costs — honestly

- Modelling is precise work: exact quantities, every output accounted for
- Today it needs a Rust-literate modeller (tutorials are included; a friendlier notation is on the roadmap)
- A few checks only run on the full build, not live in the editor
- Not yet covered: tolerances, schedules and durations, calibration against the real world

What this is *not*

- **Not simulation** — it does not predict performance; it proves the plan is internally consistent
- **Not a replacement for judgement** — it is a tireless reviewer of the bookkeeping
- **Not a silver bullet** — a model can be consistent and still wrong about the world; validating against reality is on the roadmap

Where this goes

- A ladder of case studies: from a pot of tea to multi-site production
- Generated outputs: diagrams at every level of detail; work instructions people can follow
- Analysis tools that point out gaps and weak style in a model
- A dedicated notation (a DSL), so engineers write models without writing Rust

Summary

- Plans become models; models are audited like accounts
- Contradictions fail loudly at design time, in plain language
- Requirements are traceable, tested and reported automatically
- Evidence-first: every claim measured, every limitation documented
- **The ask:** which of your processes should we model first?